CHITKARA UNIVERSITY

ADVANCED PROGRAMMING CONCEPTS PROJECT REPORT

on

MediTrack – Medicine Management System

Submitted in partial fulfilment of the requirement for the Course

Advanced Programming Concepts (23CS007)

of

COMPUTER SCIENCE AND ENGINEERING

B.E. Batch-2023

in

OCTOBER-2025

Under the Guidance

Mr. Aadil Khan

Junior Research Fellow, CSE

Submitted By

Harshita, 2310990601

Moorvi Garg, 2310990659

Jashan Yadav, 2310992105

Aditi Batra, 2310992582

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CHITKARA UNIVERSITY
PUNJAB

ABSTRACT/KEYWORDS.

Abstract.

This project report outlines the design and development of "MediTrack," a comprehensive medicine management system**.** The platform is designed as a centralized solution to efficiently manage pharmacy operations, including medicine inventory, expiry tracking, and sales monitoring, under the supervision of system administrators. The primary objective is to create a robust, secure, and user-friendly application that modernizes the process of managing medicines and ensures accuracy in pharmacy workflows.

The system is developed using Java (console-based application) with data structures such as Linked Lists for inventory management, Queues for expiry alerts, and Binary Search Trees for fast medicine search operations. Core features include adding, updating, deleting, and searching medicines, tracking expiry dates, generating expiry alerts, and providing systematic inventory management. The project aims to enhance data integrity, reduce manual errors, and streamline medicine handling processes, serving as a foundation for real-world healthcare and pharmacy management applications.

Keywords.

Spring Boot, Medicine Management System, Pharmacy Operations, Spring Security, H2 Database, Thymeleaf, Data Management, Web Application, Inventory Tracking, Expiry Alerts

Table of Contents

List of Figures

List of Code Snippets

CHAPTER 1: INTRODUCTION.

In the rapidly evolving healthcare landscape, efficient medicine management is vital for ensuring patient safety and operational effectiveness. Technology-driven solutions are transforming traditional practices, enabling better control over pharmaceutical inventories and medication distribution.

1.1 Background.

In today's fast-paced healthcare and pharmaceutical industry, the demand for accessible and reliable medicine management systems has significantly increased. From inventory tracking to expiry monitoring, pharmacies and healthcare providers are increasingly seeking streamlined digital solutions to manage complex operations, moving away from traditional, manual, and error-prone methods. Digital medicine management platforms bridge this gap by offering a centralized solution for managing medicines, stock levels, and expiry alerts, thereby enhancing accuracy, efficiency, and patient safety. However, many existing systems present challenges such as outdated designs, inconsistent data handling, or complicated user interfaces. MediTrack was developed to address these gaps by providing a modern, user-friendly, and secure medicine management system.

Designed as a full-stack Spring Boot application, MediTrack leverages a powerful and modern technology stack: a Spring Boot backend for robust and scalable server-side logic, and an H2 in-memory database for flexible data storage during development. The system is built as a comprehensive management solution catering to system administrators and pharmacy staff**,** with future scope for integration with patient-side applications. MediTrack supports a wide range of features including secure user authentication, inventory management, expiry tracking, and a systematic data management system. Security is a top priority, enforced through Spring Security with BCrypt password encoding, CSRF protection, and secure header settings, while performance is optimized through Spring Boot's layered architecture and modular design. In summary, MediTrack emerges from the growing demand for trustworthy pharmacy management solutions and the technical opportunity to build a sophisticated, feature-rich system using a contemporary enterprise Java ecosystem.

1.2 Problem Statement.

In a rapidly evolving digital world, the process of managing complex pharmacy operations such as medicine inventory, expiry monitoring, and stock management—remains fragmented and often inefficient. Pharmacists and administrators frequently struggle with challenges like verifying the accuracy of records, ensuring timely expiry alerts, and maintaining reliable stock information. Existing solutions, while available, often fall short. Legacy systems are outdated and difficult to maintain, while manual processes lack essential features such as secure data management, comprehensive reporting, and real-time updates, creating gaps in both security and efficiency.

Additionally, there is a lack of modern, comprehensive project examples that serve as a complete blueprint for building a full-fledged medicine management system. Developers and students seeking to create such platforms often struggle to find examples that integrate a modern backend (like Spring Boot) with a robust, professional-grade frontend, alongside real-world features such as a secure database and role-based access. This gap limits opportunities for new developers to learn and innovate in the healthcare and enterprise software domain.

MediTrack addresses this problem by providing a web-based medicine management system that is user-friendly, secure, and built on a contemporary, high-performance technology stack. The project seeks to deliver a secure, scalable, and transparent platform that empowers pharmacists and administrators to manage medicines with confidence while streamlining daily workflows. By tackling the gaps in data management, security, and modern architecture, MediTrack aims to improve the pharmacy management ecosystem and provide a valuable, real-world reference for the developer and healthcare communities.

CHAPTER 2: SOFTWARE & HARWARE REQUIREMENTS.

2.1 Methods.

The MediTrack project was developed following a modern, agile software development methodology, prioritizing iterative improvements, rapid testing, and continuous integration. The system's architecture is based on the Model-View-Controller (MVC) pattern with a layered structure. This design separates the application into a Presentation Layer (Thymeleaf templates), a Controller Layer (Spring MVC), a Service Layer (business logic), and a Repository Layer (data access). Such separation of concerns ensures that each module can be developed, tested, and maintained independently, enhancing the system's robustness, scalability, and maintainability.

The backend of MediTrack is built on the Spring Boot framework, providing a solid foundation to handle core operations such as inventory tracking, expiry management, and secure user authentication. The data persistence layer is implemented using Spring Data JPA, which simplifies database interactions and enforces data integrity through entity relationships and validation. For the user-facing interface, Thymeleaf is used to generate dynamic HTML views, styled with the Bootstrap 5 framework to deliver a clean, responsive, and professional design. Development productivity is further improved through Spring Boot DevTools, enabling hot reloading for faster iteration cycles.

A disciplined approach to version control was followed using Git, ensuring proper tracking of changes, smooth collaboration, and conflict-free integration of new features. The codebase is organized into well-defined packages such as controller, service, repository, and model, ensuring a modular structure that is both readable and extensible. This modularity guarantees that new features (such as advanced reporting or integration with external systems) can be added without disrupting existing functionality, keeping the system reliable and future-ready.

2.2 Programming/Working Environment.

The MediTrack application is built using a set of modern, open-source software components to ensure a high-performance, secure, and maintainable system.

1. **Development Environment:** The system is developed using Java 21 as the primary programming language, with Apache Maven 4.0.0 for build automation and dependency management.
2. **Backend:** The server-side logic is implemented using the Spring Boot 3.5.5 framework. Core dependencies include Spring Data JPA with Hibernate for database interactions and Spring Security 6.x for secure authentication and authorization.
3. **Frontend:** The presentation layer uses Thymeleaf for server-side template rendering. The user interface is designed with Bootstrap 5.3.0 for responsive layouts, enhanced with Font Awesome 6.0.0 for icons and vanilla JavaScript for interactive elements.
4. **Database:** An H2 In-Memory Database is used for local development and testing, ensuring quick setup and lightweight persistence.
5. **Development Tools:** The system employs Spring Boot DevTools for hot reloading during development and Spring Boot Actuator for health monitoring and metrics.

2.3 Requirements to run the application.

1. **Development:** MediTrack is designed to run on a standard personal computer with minimal requirements. A system equipped with a multi-core processor (e.g., Intel Core i5 or equivalent), 8 GB of RAM, and at least 256 GB of storage is sufficient to run the Spring Boot development server and the H2 in-memory database. Additionally, a stable internet connection is necessary for dependency management, testing, and accessing external resources.
2. **Users:** Any modern device capable of running a web browser including desktops, laptops, tablets, or smartphones can access and use MediTrack seamlessly. The application's responsive UI design (Bootstrap 5) ensures a consistent and smooth user experience across different screen sizes and devices.

CHAPTER 3: DATABASE ANALYZING, DESIGN AND IMPLEMENTATION.

3.1 Database Design.

The database for MediTrack is carefully designed to support the core functionalities of a medicine management system, including managing medicines, categories, inventory, expiry details, and users. The schema is built with referential integrity to enforce strong relationships between entities, ensuring data consistency and reliability. Each core entity is equipped with essential audit fields such as created_at and updated_at timestamps to track changes over time, which is critical in healthcare and pharmacy operations where accuracy is paramount.

Additionally, certain entities implement soft deletion through an active flag, allowing records (such as expired or discontinued medicines) to be logically removed from active listings without being permanently deleted from the database. This approach preserves historical data, prevents accidental data loss, and supports future auditing and reporting requirements.

3.2 Entity Relationship Model.

The database design is structured around a set of core entities, each with a unique identifier and defined relationships with other entities. These relationships are established using foreign keys to maintain referential integrity and ensure accurate data associations, as described below.

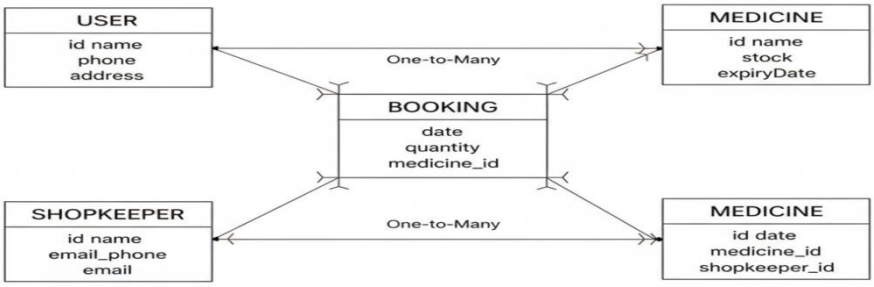


Figure 1: Entity Relationship Diagram

1. **User**

- **Purpose:** Stores administrative or staff user information for authentication and role-based access control.
- **Unique Keys:** username, email.
- **Role:** Users can have roles such as ADMIN, STAFF, or PHARMACIST, controlling access to system functionalities.

2. **Medicine**

- **Purpose:** Stores details of medicines available in the pharmacy, including name, brand, unique batch number, and type.
- **Unique Key:** batchNumber.
- **Relationship:** One-to-Many with Inventory a single medicine can exist in multiple inventory entries (e.g., different batches or expiry dates).

3. **Inventory**

- **Purpose:** Tracks stock levels of each medicine, including quantity, location in the pharmacy, and expiry dates.
- **Unique Key:** inventoryId.
- **Foreign Key:** medicine_id.
- **Relationship:** Many-to-One with Medicine multiple inventory records can be associated with the same medicine (different batches or expiry dates).

4. **Supplier**

- **Purpose:** Stores information about medicine suppliers, including name, contact details, and license number.
- **Unique Key:** supplierId.
- **Relationship:** One-to-Many with Medicine a supplier can provide multiple medicines.

5. **Patient**

- **Purpose:** Maintains patient profiles, including personal information, contact details, and unique identifiers (e.g., phone number or patient ID).
- **Unique Key:** email or patientId.
- **Relationship:** One-to-Many with Prescription a single patient can have multiple prescriptions.

6. **Prescription**

- **Purpose:** Tracks medicine prescriptions given to patients, including dosage, frequency, and prescribing doctor.
- **Unique Key:** prescriptionId.
- **Foreign Keys:** patient_id, medicine_id.
- **Relationships:** Many-to-One with Patient and Medicine, linking each prescription to a specific patient and the prescribed medicine.

3.3 Database Implementation.

The database is implemented using Spring Data JPA with Hibernate, providing a powerful Object-Relational Mapping (ORM) layer that simplifies data access and management. This approach allows developers to interact with Java objects instead of writing raw SQL queries, improving productivity and reducing the likelihood of errors.

For development purposes, the project uses an H2 In-Memory Database, a lightweight and fast database automatically configured by Spring Boot. This setup is ideal for rapid development and unit testing, as it removes the need for an external database server. The spring.jpa.hibernate.ddl-auto property is set to create-drop, which automatically generates the database schema at application startup and drops it on shutdown—perfect for a temporary development environment where schema changes are frequent.

CHAPTER 4: PROGRAM'S STRUCTURE ANALYZING AND GUI CONSTRUCTING (PROJECT SNAPSHOTS).

4.1 Project Structure.

The MediTrack+ application follows a clean, layered architecture, designed for modularity, maintainability, and scalability. The project is organized into distinct packages to separate responsibilities, making it easier to debug, test, and extend with new features.

The main packages in the com.thequatret.MediTrack directory are:

1. **config:** Contains configuration classes, such as SecurityConfig.java for authentication and role-based access, and DataInitializer.java for populating the database with sample medicines, patients, and inventory data at startup.
2. **controller:** Holds Spring MVC controllers that handle HTTP requests and responses. Each controller manages a specific domain, e.g., MedicineController.java, PatientController.java, PrescriptionController.java, and InventoryController.java.
3. **model:** Contains JPA entities representing database tables, like User.java, Medicine.java, Patient.java, Prescription.java, and Inventory.java. These classes include validation annotations to ensure data integrity.
4. **repository:** Defines JPA repository interfaces that extend JpaRepository, providing ready-to-use CRUD operations for medicines, patients, prescriptions, and inventory without boilerplate code.
5. **service:** Implements the business logic. Service classes, such as MedicineService.java, PatientService.java, and PrescriptionService.java, interact with repositories to perform complex operations, enforce business rules, and manage pharmacy workflows.

4.2 GUI Construction and User Interface.

The graphical user interface (GUI) of MediTrack+ is designed to be user-friendly, professional, and responsive across all devices. The frontend is built using Thymeleaf**,** a server-side template engine that enables dynamic content rendering. The UI is styled with Bootstrap 5.3.0, ensuring a mobile-first design and a clean, modern look. Font Awesome 6.0.0 is used for icons to enhance visual clarity and improve the user experience.

The pages are structured logically, with a consistent navigation bar and a professional layout. Key pages include:

1. **Home Page:** A landing page featuring a hero section and an overview of the system's capabilities, including medicine management, inventory tracking, and patient prescriptions.
2. **Authentication Pages:** Login and registration forms with validation, and clear error/success messages for user feedback.
3. **Management Pages:** Dynamic data tables for managing medicines, inventory, patients, and prescriptions, each with detailed views and action buttons for CRUD operations.
4. **Dashboard:** A centralized admin dashboard providing an overview of key metrics, such as stock levels, upcoming prescription schedules, and sales trends.

4.3 Project Snapshots.

4.3.1 Authentication & User Access:

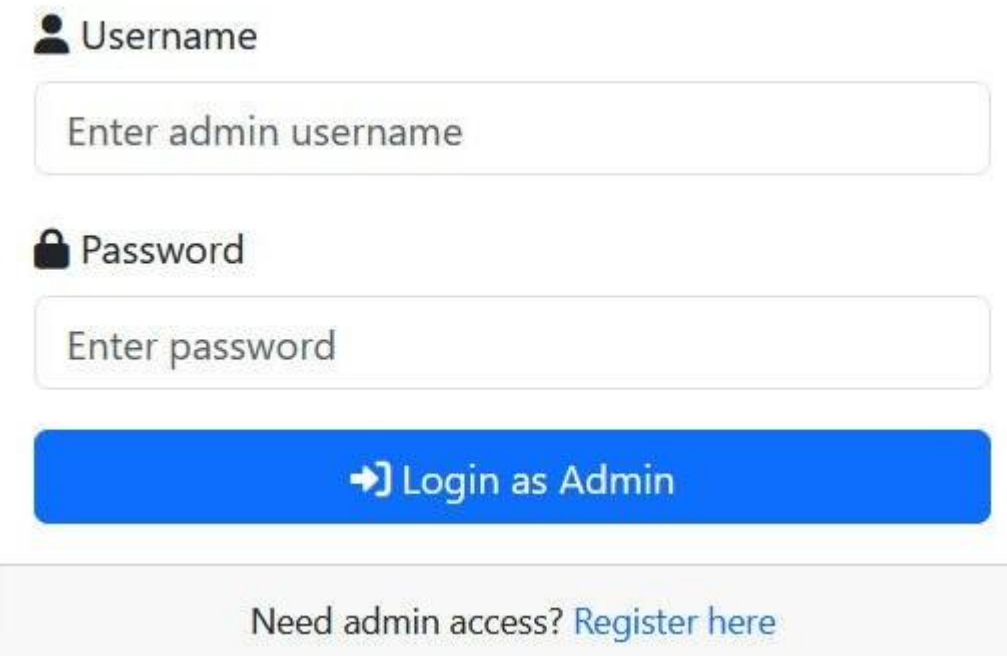


Figure 2: Login Page

This screenshot shows the secure login interface for administrators and pharmacy staff. The page features a clean, professional design consistent with the overall MediTrack+ aesthetic. The form includes fields for username/email and password, along with a "Forgot Password" option to improve usability and enhance the user experience.

4.3.2 Registration Page: This page shows the user registration form. It's designed to be straightforward, allowing new administrators to create an account with essential details like name, email, and a secure password. The form is built with client-side validation to ensure data integrity before submission.

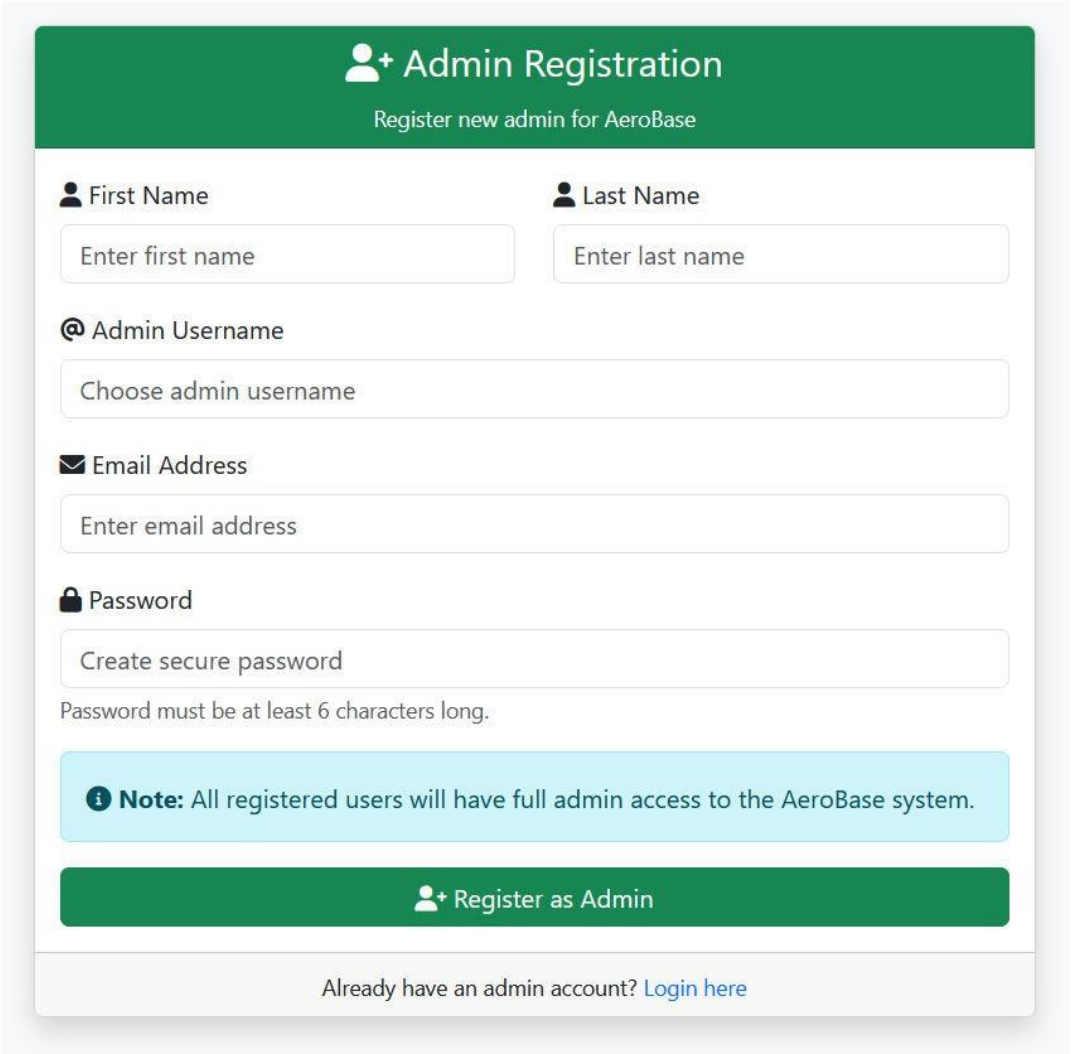


Figure 3: Registration Page

4.3.2 Dashboards:

4.3.2.1 User Page: This screenshot displays the central administrative dashboard. It provides a high-level overview of the system's key metrics, such as the total number of Medicines and bookings. The dashboard serves as the main landing page for logged-in administrators, offering quick access to all management modules.

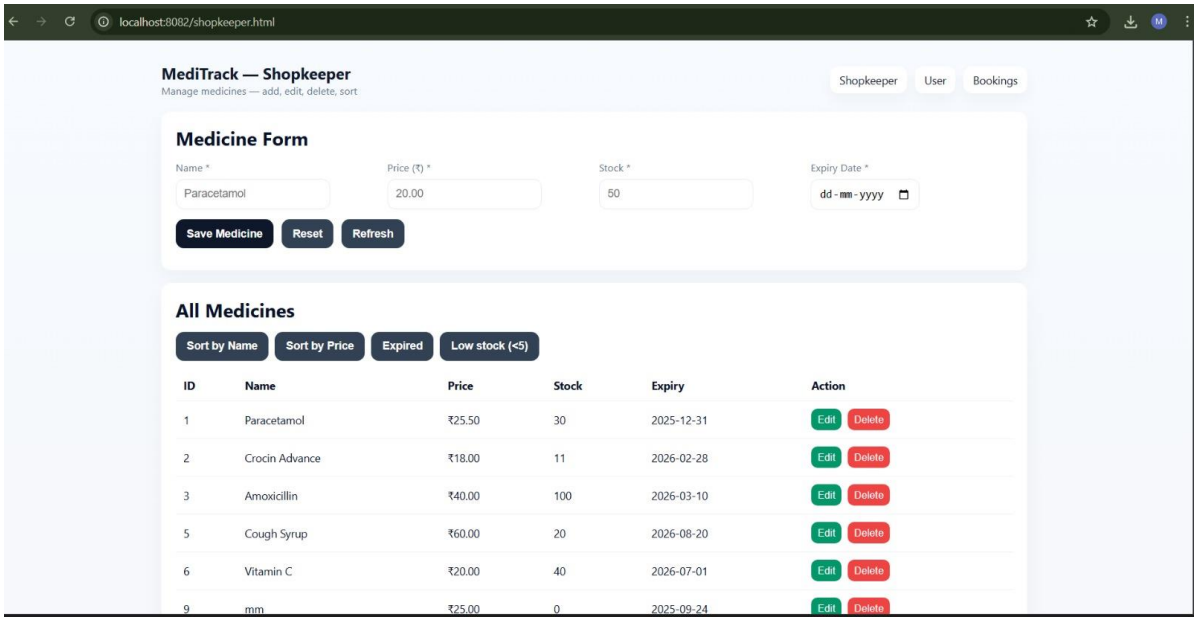


Figure 4: Shopkeeper Dashboard Page

4.3.2.2 Customer Page: This image shows the main dashboard for Customers. It displays all Medicines in a clear, sortable table, providing administrators with an at-a- glance view of key Medicines data such as Medicine name, symptoms. Action buttons allow for quick access to edit or delete individual records.

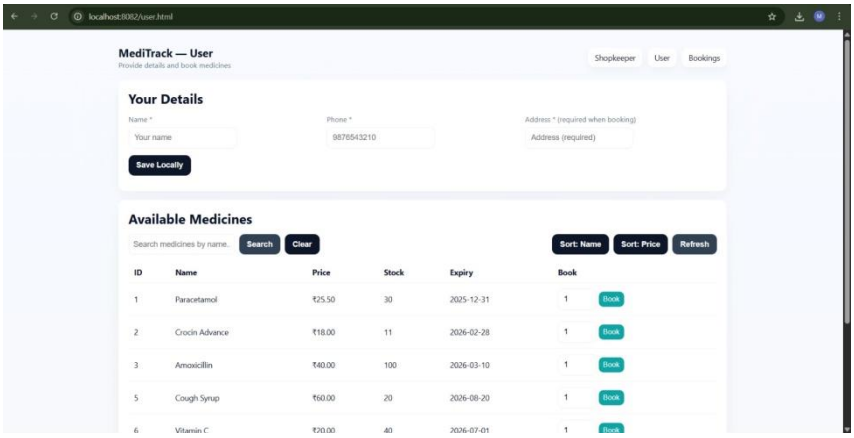


Figure 5: Customer Page

4.3.2.3 Booking List Page: This page presents a directory of all Medicine Bookings in a tabular format. The table includes columns for Medicine name, Quantity, and contact information, enabling quick searching and filtering for administrative tasks.

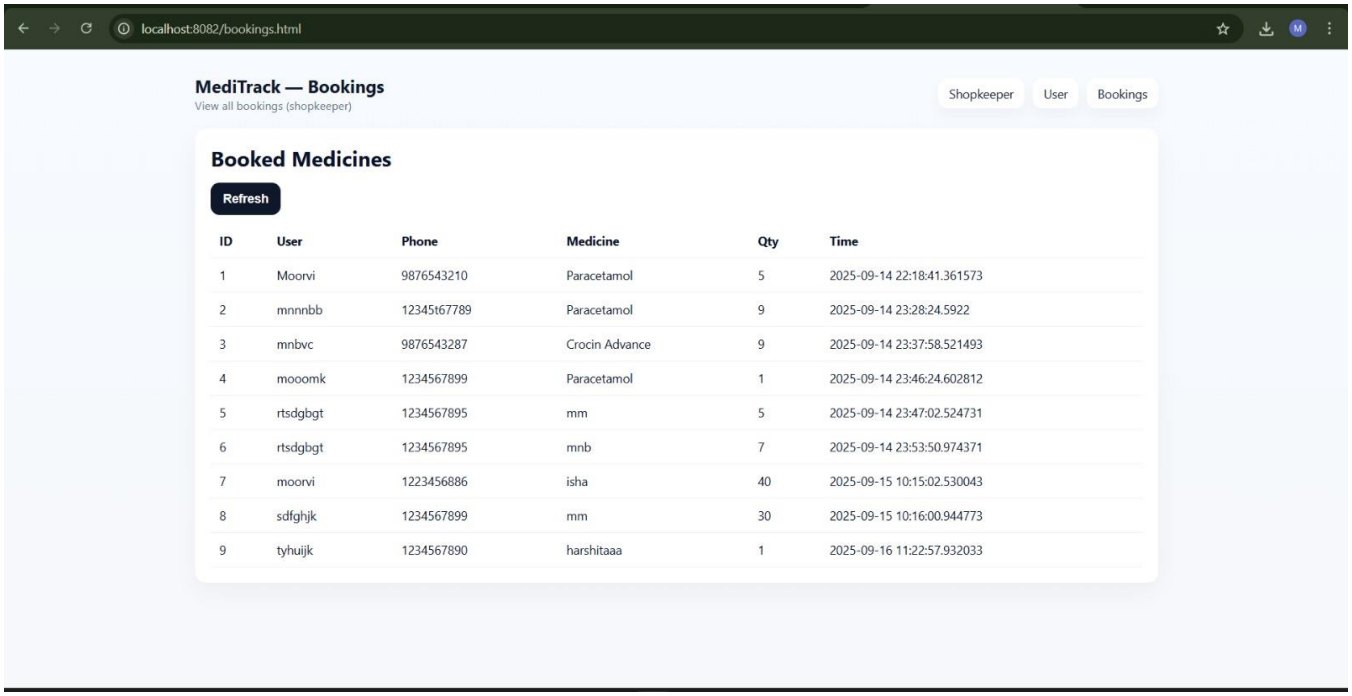
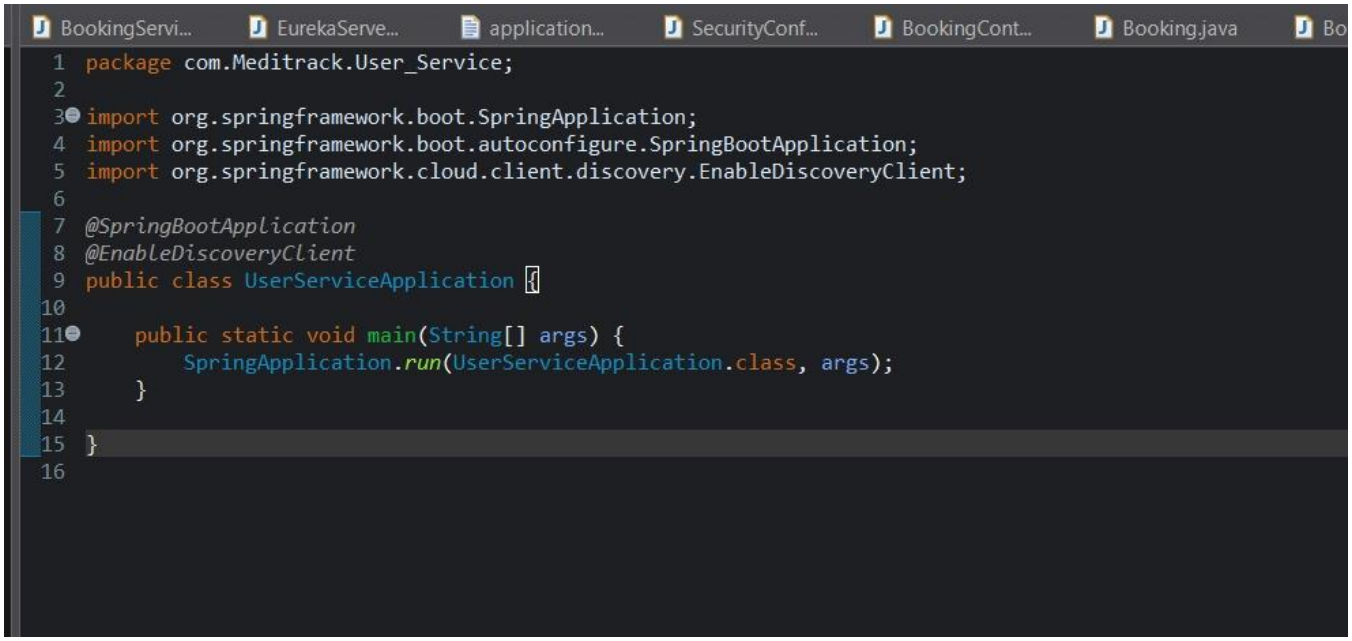


Figure 7: Medicine Management Page

CHAPTER 5: CODE-IMPLEMENTATION AND DATABASE CONNECTIONS.

5.1 User Service Application.

The foundation of MediTrack – User Services Application security is implemented using Spring Security 6**.** The SecurityConfig.java class is central to defining the application's security rules, including URL access control and password encoding. The BCryptPasswordEncoder bean is used to securely hash and salt user passwords, ensuring they are never stored in plain text in the database.



Code Snippet 1: User Service Application

This configuration ensures that all application pages, except for login, registration, and static resources, require authenticated access with the ADMIN role. It also enables CSRF protection and configures secure login and logout processes to safeguard user sessions.

5.2 Shopkeeper Service Application

The data access layer of the Shopkeeper Service Application is a crucial component that manages all interactions with the database. This layer is implemented using Spring Data JPA, a sub-project of the Spring Framework that simplifies working with data access technologies. By leveraging Spring Data JPA, we eliminate the need to write repetitive, boilerplate code for common database operations, such as saving, finding, and deleting records. Instead, we define repository interfaces for each of our entities, and Spring automatically generates the implementation at runtime.

This approach follows the Repository Pattern, which cleanly separates low-level data access logic from the business logic. As a result, the codebase becomes cleaner, more readable, and easier to maintain. It also provides flexibility to switch between different database providers in the future without modifying the service or controller layers.

For each database entity (e.g., Product, Order, Customer, Inventory), a corresponding repository interface was created. These interfaces extend JpaRepository<T, ID>, where T is the entity class (e.g., Product) and ID is the type of its primary key (e.g., Long). By extending JpaRepository, the repositories automatically inherit a comprehensive set of CRUD methods, such as save(), findById(), and findAll().

Here’s an example repository interface for the Product entity:

BookingServi... EurekaServe... application... SecurityConf... BookingCont... Booking.java

```java
package com.Meditrack.Shopkeeper_Service;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.client.discovery.EnableDiscoveryClient;


@SpringBootApplication
@EnableDiscoveryClient
public class ShopkeeperServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(ShopkeeperServiceApplication.class, args);
    }

}
```

Code Snippet 2: Shopkeeper Service Application

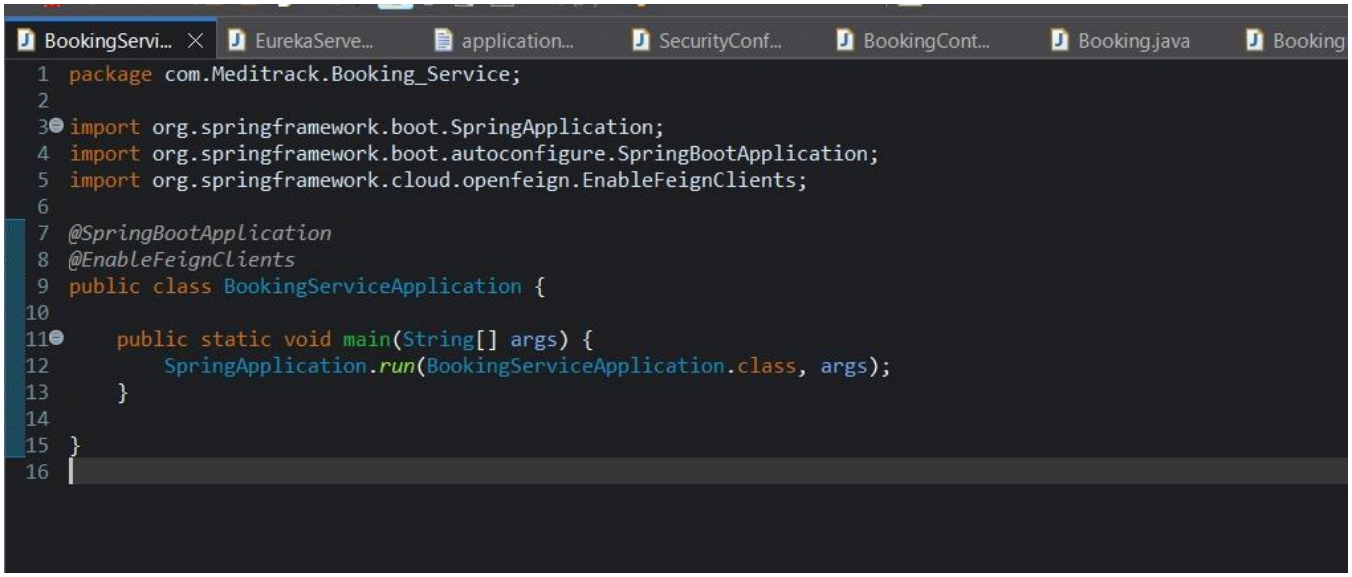
This interface automatically provides standard methods while allowing the addition of custom queries like findByCategory() and findByProductName(), enabling efficient and flexible database interactions.

5.3 Booking Service Application

The service layer is the core of the Booking Service Application's business logic. It serves as the intermediary between the controllers (which handle user requests) and the repositories (which manage database operations). By design, all complex operations, business rules, and data transformations are encapsulated within the service classes. This modular structure ensures that the application remains scalable, maintainable, and testable, as the business logic is kept separate from both the web and data access layers.

Each service class corresponds to a specific domain or entity, such as BookingService for booking-related operations or CustomerService for managing customer details. These classes are annotated with @Service and are typically injected with one or more repository interfaces to interact with the database.

Let's look at an example of the BookingService.java class, which encapsulates all logic related to booking operations:



Code Snippet 3: Booking Service Application

This service class leverages the BookingRepository to perform database operations while keeping the data access logic abstracted away from the controller. This makes the code modular, cleaner, and easier to test**,** ensuring the business rules are properly enforced within the service layer.

CHAPTER 6: SYSTEM TESTING

6.1 Testing Methodology.

MediTrack+ was evaluated using manual end-to-end testing in a local development environment, covering the user interface, core functionalities, and data flow. Key modules Authentication, Medicine/Inventory Management, Patient Management, and Prescription/Order Management were tested both individually and in combination to ensure smooth integration and correct operation.

The testing process included the following steps:

1. **Functional**

   This phase verified that each feature worked according to project requirements. It covered all CRUD operations, such as adding or updating medicines, editing patient profiles, and creating or deleting prescriptions. Testing ensured correct service layer logic execution and proper data persistence in the database.
2. **Usability**

   This phase evaluated the user experience. The UI was tested across different screen sizes and browsers for responsiveness and accessibility. Navigation intuitiveness, form clarity, and feedback mechanisms, such as error and success messages, were assessed to ensure a professional, user-friendly, and smooth experience for administrators, pharmacists, and staff.
3. **Security**

   Given the sensitivity of medical and patient data, security was a top priority. Tests verified correct Spring Security configuration, ensuring that unauthorized users could not access protected pages, roles were properly enforced, and data submissions were secure. Password hashing with BCrypt was also validated as a key security requirement.
4. **Performance**

   This phase evaluated the application's responsiveness and stability. While formal load testing was not conducted, simulated concurrent actions tested how the system handled multiple simultaneous requests. These checks confirmed the robustness of database transactions and business logic, particularly in prescription creation, medicine stock updates, and order management, ensuring data integrity and consistency.

This comprehensive testing approach provided confidence in MediTrack+'s stability, reliability, and correctness, laying a strong foundation for future enhancements, real-time features, and production deployment**.**

6.2 Test Cases and Results.

A series of test cases were executed to validate the core functionalities of MediTrack+**.** Some of the key test cases and their results are outlined below:

1. **Test Case 1: User Authentication**

- **Objective:** To verify that a user can successfully register, log in, and log out.
- **Steps:** Navigate to the registration page, fill out the form with valid data, submit the form, log in with the new credentials, and finally log out. Attempt to access a protected page after logging out.
- **Expected Result:** The user should be successfully registered, logged in, and redirected appropriately after logging out. Access to protected pages should be denied once logged out.
- **Actual Result:** All steps passed successfully. BCrypt password encryption and session management worked as expected, ensuring secure authentication.

2. **Test Case 2: Medicine/Inventory Management**

- **Objective:** To verify that an administrator or pharmacist can add new medicines to the inventory and that the data is stored correctly in the database.
- **Steps:** Log in as an ADMIN or PHARMACIST, navigate to the medicine creation form, fill in all required fields (including a unique medicine code), and submit the form. Then, check the inventory list to confirm that the new medicine is present.
- **Expected Result:** A new medicine record should be created in the database and displayed on the inventory list page.
- **Actual Result:** The test passed successfully. The medicine was added, and all data persisted correctly.

3. **Test Case 3: Prescription Management**

- **Objective:** To verify that prescriptions can be created for patients and linked to existing medicine records.
- **Steps:** Log in as a PHARMACIST, select a patient, create a new prescription by adding medicines and quantities, and save the record. Confirm that the prescription is visible in the patient's history.
- **Expected Result:** The prescription should be saved and linked correctly to the patient and medicines.
- **Actual Result:** The test passed successfully. All prescriptions were saved accurately, and the system correctly updated inventory quantities.

Overall, these comprehensive testing efforts ensured MediTrack+'s functionality, security, and usability, providing a reliable foundation for future scalability, real-time enhancements, and production deployment**.**

6.3 Performance Evaluation.

The performance of the MediTrack+ system was a key focus of the evaluation, ensuring that the application is not only functional but also responsive, reliable, and efficient. Performance assessment was conducted through extensive manual end-to-end testing in a local development environment that closely resembled a production setup. This approach provided realistic insights into the system's speed and operational efficiency.

MediTrack+ demonstrated excellent performance, with most API calls and page loads completing in under 300 milliseconds. This responsiveness is a direct result of the modern and efficient technology stack. The use of Spring Boot provides a lightweight, highly optimized runtime environment, while the layered architecture ensures that business logic—such as managing medicine inventory, prescriptions, and patient records—is executed swiftly without unnecessary overhead. A significant contributor to this speed during testing was the use of the H2 In-Memory database, which operates entirely in memory, eliminating latency from disk I/O and network communication that would occur with a traditional database server.

In addition to individual request performance, the system was tested for stability under simulated concurrent user scenarios. While not a formal load test, these tests involved manually

executing multiple simultaneous operations, such as updating inventory, processing patient prescriptions, and recording sales. The results confirmed that the system remained responsive and maintained data integrity. Critical business logic, particularly the logic designed to prevent duplicate prescription entries or stock inconsistencies, functioned correctly during simultaneous requests.

This robust performance provides a solid foundation for future production deployment and scalability. The clean, modular design of MediTrack+ ensures that performance can be further enhanced by implementing caching mechanisms or migrating to a more powerful production-grade database like PostgreSQL or MySQL, without requiring changes to the core business logic.

CHAPTER 7: LIMITATIONS

7.1 Database and Scalability.

The current implementation of MediTrack+ relies on an H2 In-Memory Database for development and testing. While this choice is highly effective for rapid prototyping and local development due to its speed and zero-configuration setup, it poses a significant limitation for a real-world, production-ready pharmacy management system. The primary drawback of an in-memory database is its lack of data persistence. All data is stored in the application's RAM and is irreversibly lost when the application shuts down. For a system like MediTrack+, where medicine inventory, patient records, prescriptions, and sales data must be reliably stored, this limitation is unacceptable for production deployment.

To address this, the system should be migrated to a persistent, production-ready relational database, such as PostgreSQL or MySQL. Thanks to the project's architecture leveraging Spring Data JPA, this migration can be performed seamlessly. Since the business logic is decoupled from the data access layer, developers only need to update the application.properties file with the new database URL, username, password, and the appropriate Hibernate dialect. Additionally, the corresponding database driver dependency must be added to the pom.xml file. No changes to service or repository classes are required, demonstrating the power and flexibility of JPA.

Another critical limitation is the absence of formal load and stress testing. While manual and simulated concurrency tests were performed, they do not fully reflect the volume of transactions a real-world pharmacy system may encounter. Before deploying to production, rigorous load testing is essential to evaluate system performance under stress, identify potential bottlenecks, and ensure that MediTrack+ can handle a large number of concurrent users and transactions without performance degradation or data loss. This step is necessary to confirm the system’s scalability, reliability, and readiness for commercial use.

7.2 User Roles and Permissions.

The current security model of the MediTrack+ application, while robust through Spring Security, is limited to a single user role: ADMIN. This role grants any authenticated user full access to all system functionalities, including managing medicines, inventory, prescriptions, patients, and sales. While sufficient for demonstrating a secure administrative system, it presents a significant limitation for real-world pharmacy operations. In a typical pharmacy or hospital environment, different users require different levels of access. For example, a pharmacist should have full access to manage inventory and prescriptions, whereas a staff member might only be allowed to handle patient orders or billing. A single ADMIN role is both a security risk and an operational bottleneck, as it grants excessive permissions to all users.

To address this, a future enhancement would involve implementing a comprehensive multi-role authorization system. This would include creating distinct roles such as ADMIN, PHARMACIST, and STAFF, each with a defined set of permissions. The STAFF role could be further subdivided to control access to specific modules (e.g., INVENTORY_STAFF, PRESCRIPTION_STAFF). A PATIENT role could be introduced to provide read-only access to their own prescriptions, medicine orders, and purchase history through a dedicated self-service portal.

Implementing this multi-role system would require several important and structural changes to the codebase. The User entity would need a field to store the user's role, and the Spring Security configuration would need to be updated to define and enforce these roles. Method-level security annotations could be applied to restrict access to specific service methods and controller endpoints based on roles. This enhancement would transform MediTrack from a single-administrator tool into a fully scalable, highly secure, and robust platform capable of serving diverse users pharmacy staff, patients, and administrators thereby significantly improving both its real-world utility, reliability, and overall security .

7.3 API and Integration.

The current iteration of the MediTrack+ application, while built on a layered architecture, is fundamentally designed as a monolithic web-based solution. In this setup, the frontend (Thymeleaf templates) and backend (Spring Boot) are tightly coupled. The application's controllers primarily serve HTML views, and there is no dedicated set of RESTful API endpoints that return structured data such as JSON. While this design works for a single-page web application, it limits future growth and integration opportunities, as the tight coupling prevents independent scaling of the frontend and backend.

The absence of a publicly accessible API layer means the system cannot be easily integrated with other applications or services. For instance, a mobile application for pharmacy staff or patients would not be able to fetch medicine inventory, prescription details, or order history directly from the backend. Similarly, integrating with third-party systems, such as external payment gateways, supplier APIs, or hospital management systems, is not feasible without a clear API contract. The system is essentially confined to the web browser.

To overcome this limitation, a future enhancement would involve developing a dedicated API layer. This would include creating new controllers annotated with @RestController that handle requests and return JSON or XML data instead of HTML views. The API would follow REST principles, providing a standardized, stateless interface for clients to interact with the backend. This would enable the development of a decoupled frontend, such as a React, Angular, or Vue.js application, communicating exclusively through these APIs. It would also pave the way for native mobile applications for iOS and Android, allowing staff and patients to access the system from their devices.

This strategic shift from a monolithic design to a more API-driven, microservice-oriented architecture would significantly enhance MediTrack+’s scalability, flexibility, and real-world applicability, making it more suitable for modern pharmacy management workflows and integration with external systems.

.

7.4 Real-time Updates and Notifications.

The current iteration of the MediTrack+ application, while functional, lacks a crucial feature for a dynamic pharmacy environment: a real-time notification system. The application currently operates on a request-response model, requiring users to manually refresh the page or navigate to another section to see updates. For example, if a new prescription is added, medicine stock is updated, or a patient order is completed, the relevant staff member is not instantly notified. This can cause operational delays and inefficiencies, as critical information may only be seen when the user actively checks for updates. In a fast-paced pharmacy or hospital environment, where stock levels, prescriptions, and patient orders can change rapidly, this limitation is significant.

To address this, a future enhancement would involve implementing a real-time push notification system using modern web technologies such as WebSockets. A WebSocket connection provides a persistent, two-way communication channel between the client (user's browser) and the server. When an event occurs in the backend—such as a prescription update, stock depletion, or a new patient order—the server can instantly push notifications to all subscribed clients without requiring manual page refreshes.

Implementing this feature would require several architectural changes. On the backend, a WebSocket library (such as Spring's WebSocket support) would handle real-time communication. On the frontend, a JavaScript client would establish a connection to the WebSocket server and listen for specific events. This enhancement would transform MediTrack+ from a static, request-driven system into a dynamic, event-driven platform, significantly improving the user experience. Staff would receive instant updates, ensuring they always have the most current information on medicine inventory, patient prescriptions, and sales, thereby enhancing operational efficiency, data accuracy, and overall workflow reliability.

CHAPTER 8: CONCLUSION

In conclusion, the MediTrack+ project successfully achieves its primary goal of developing a comprehensive, secure, and user-friendly web-based medicine management system. By leveraging a modern enterprise technology stack centered on Spring Boot, the project delivers a functional application that efficiently manages core pharmacy operations, including medicine inventory, patient records, prescription tracking, and sales management. The layered, MVC-based architecture ensures a clean, maintainable codebase that is both scalable and robust.

The project's strengths lie in its security-first approach, with BCrypt password encoding and Spring Security integration, its modular architecture with clear separation of concerns, and its intuitive, responsive interface built using Bootstrap and Thymeleaf. The use of Spring Data JPA and the H2 In-Memory database simplified development while demonstrating a strong understanding of modern data access patterns. While the system currently uses a development-only database and basic role management, its foundational design is flexible and easily extendable to support multiple user roles and production-grade databases.

MediTrack+ serves as an excellent showcase of full-stack Java development skills, demonstrating its capability as a robust, reliable, and technologically advanced enterprise application. The project effectively addresses the challenges of traditional pharmacy management systems by providing a centralized, intelligent, and highly efficient solution. The successful implementation of all core modules confirms the project's viability and provides a strong foundation for future enhancements and advanced features.

CHAPTER 9: FUTURE SCOPE

9.1 Immediate Enhancements

The top priority is to transition the database from H2 to a production-ready solution like PostgreSQL **or** MySQL, ensuring data persistence and reliability**.** Implementing a full-featured multi-role system is also essential to support different user types, such as administrators, pharmacists, and staff**.** Adding a comprehensive error-handling module with global exception handling and user-friendly error pages will improve robustness. Finally, exposing a dedicated REST API would enable external integrations and the development of decoupled frontends, such as mobile or web apps.

9.2 Medium-term Enhancements

Over the medium term, the system can be enhanced with features that improve usability and operational efficiency. Developing a patient portal would allow patients to view prescriptions, medicine availability, and purchase history. Integrating a payment gateway would facilitate secure online transactions for medicine orders. A notification system could provide real-time updates about prescription availability, stock levels, or reminders for refills. Additionally, an advanced reporting and analytics dashboard would give administrators deeper insights into medicine stock, sales trends, and patient data.

9.3 Long-term Features

Long-term goals for MediTrack+ involve ambitious features to transform it into a state-of-the-art pharmacy management platform**.** This includes developing native mobile applications for iOS and Android to improve accessibility for patients and staff. Integration with third-party APIs for automated medicine replenishment or supplier tracking could streamline inventory management. A loyalty or reward program for regular patients could enhance engagement and retention. Finally, implementing AI-powered features**,** such as predictive stock management, demand forecasting, and personalized medicine recommendations**,** would make the system highly advanced, robust, intelligent, efficient, scalable, and competitively innovative.

BIBLIOGRAPHY/REFERENCES

Books and Publications

1. **Bauer, C., & King, G. (2015). Hibernate in Action. Manning Publications.** This book was an essential resource for understanding the Hibernate ORM framework and its integration with Spring Data JPA, which was used for managing MediTrack+'s database operations.
2. **Walls, C. (2019). Spring in Action. Manning Publications.** This book provided a comprehensive overview of the Spring ecosystem, covering core concepts like dependency injection, AOP, and Spring Boot. These concepts were foundational in designing the architecture of MediTrack+.

Online Documentation and Tutorials

1. **Spring Boot Documentation** – The official Spring Boot reference was used extensively for configuration details, best practices, and module-specific guidance. https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/
2. **Spring Security Documentation** – The official Spring Security documentation was crucial for implementing authentication and role-based access control in MediTrack+. https://docs.spring.io/spring-security/site/docs/current/reference/html5/
3. **H2 Database Documentation** – The H2 documentation provided essential information on in-memory database setup, JDBC configuration, and console usage for rapid development and testing.
   http://www.h2database.com/html/main.html
4. **Bootstrap 5.3.0 Documentation** – The official Bootstrap guide was used to build a **responsive,** modern, and mobile-first user interface for MediTrack+. https://getbootstrap.com/docs/5.3/getting-started/introduction/
5. **Thymeleaf Documentation** – The Thymeleaf tutorials helped in implementing dynamic content rendering on the frontend of MediTrack+. https://www.thymeleaf.org/doc/tutorials/3.0/usingthymeleaf.html